COMP90038 Algorithms and Complexity

Balanced Trees

Junhao Gan

Week 8 Lecture 15

Semester 1, 2026

Approaches to Balanced Binary Search Trees

To optimise the performance of BST search, it is important to keep the trees (reasonably) "balanced".

Instance simplification approaches: self-balancing trees

- AVL trees
- Red-black trees
- Splay trees

Representational changes: beyond binary trees

- 2–3 trees
- 2–3–4 trees
- B-trees

AVL Trees

Named after Adelson-Velsky and Landis.

Recall that we defined the height of a tree as the number of edges on the longest root-to-leaf path.

In particular, the height of an empty tree is defined as -1 and the height of a single-node tree is 0.

AVL Trees

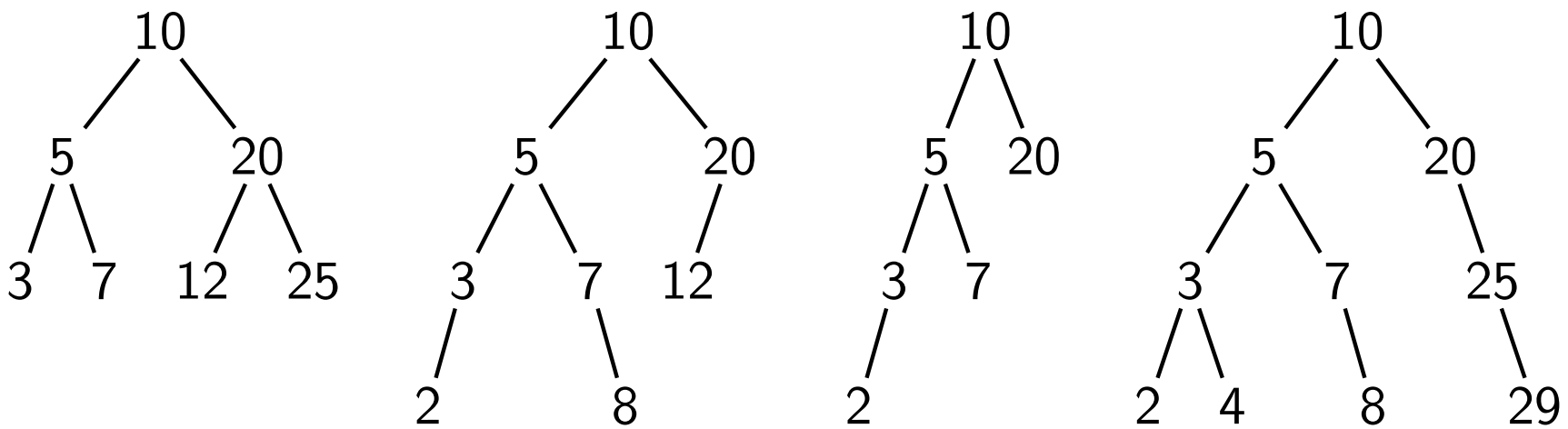
Consider a binary (sub-)tree; the balance factor of a node u is the **height difference** between its left sub-tree and its right sub-tree.

An **AVL tree** is a BST in which the balance factor of *every* node can only be either -1, 0, or 1.

The condition that requires every node to have its *absolute balance factor* at most 1 is called the **AVL Balance Condition**.

AVL Trees: Examples and Counter-Examples

Which of these are AVL trees?

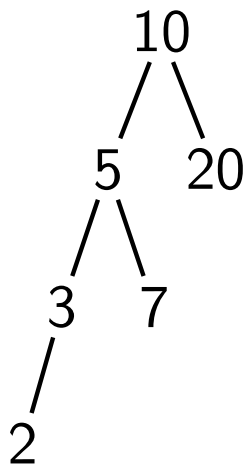


Building an AVL Tree

As with standard BSTs, a new node is always inserted as a leaf in the tree.

It is possible that after a node insertion, the AVL tree becomes *unbalanced*, in the sense that, some nodes get absolute balance factors of 2.

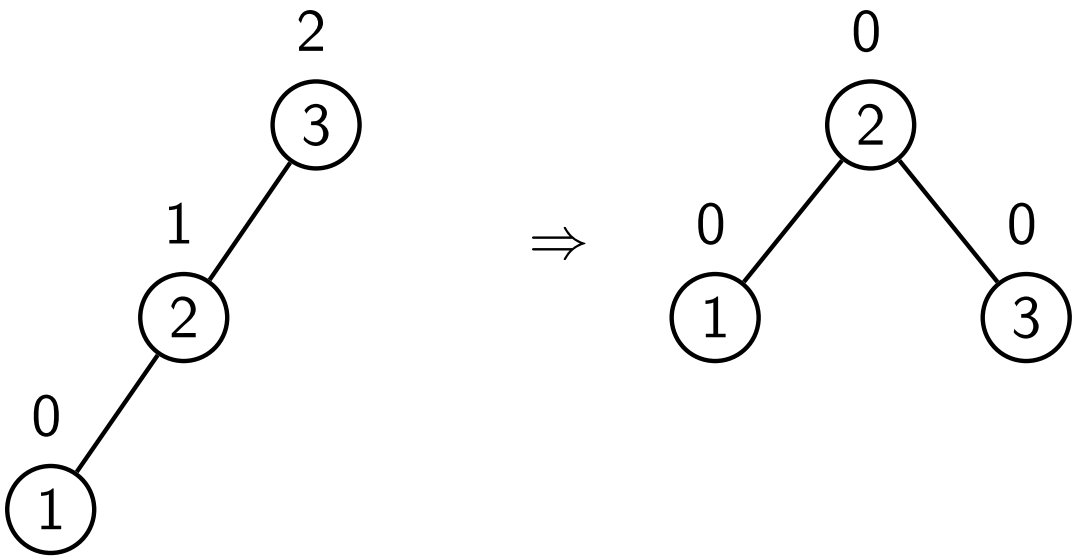
In this case, **the AVL balance condition is violated**.



The balance violation can be resolved by one or two simple, local transformations, so-called rotations.

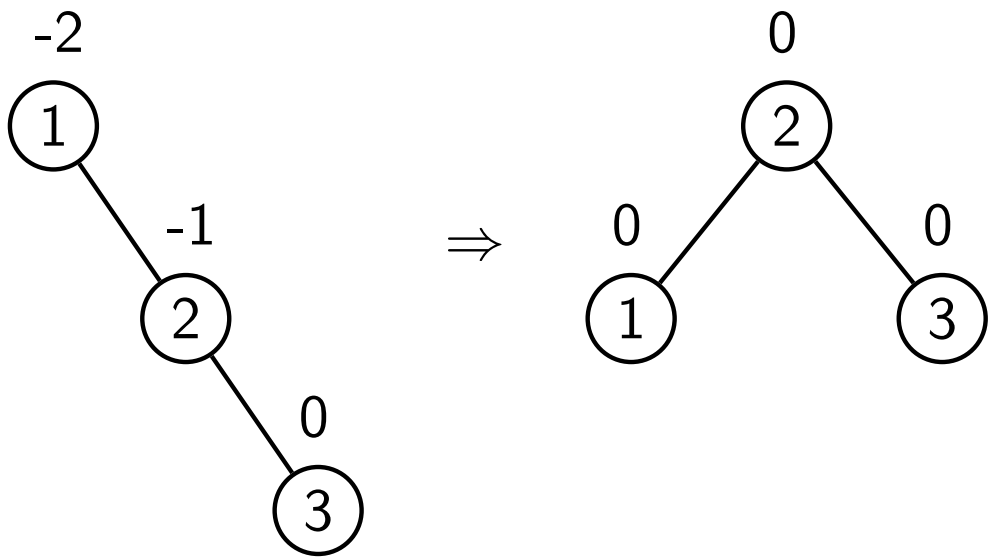
AVL Trees: R-Rotation

A right rotation (for short, R-Rotation) at node 3:



AVL Trees: L-Rotation

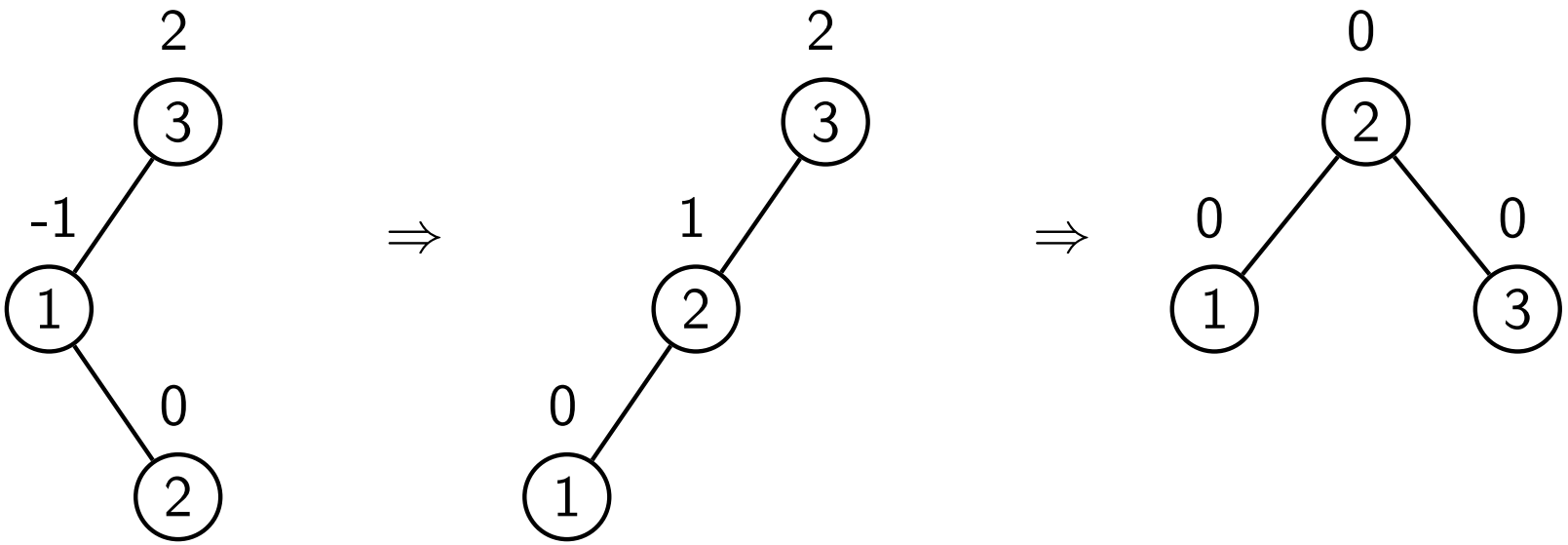
A left rotation (for short, L-Rotation) at node 1:



AVL Trees: LR-Rotation

An LR-Rotation at node 3 performs:

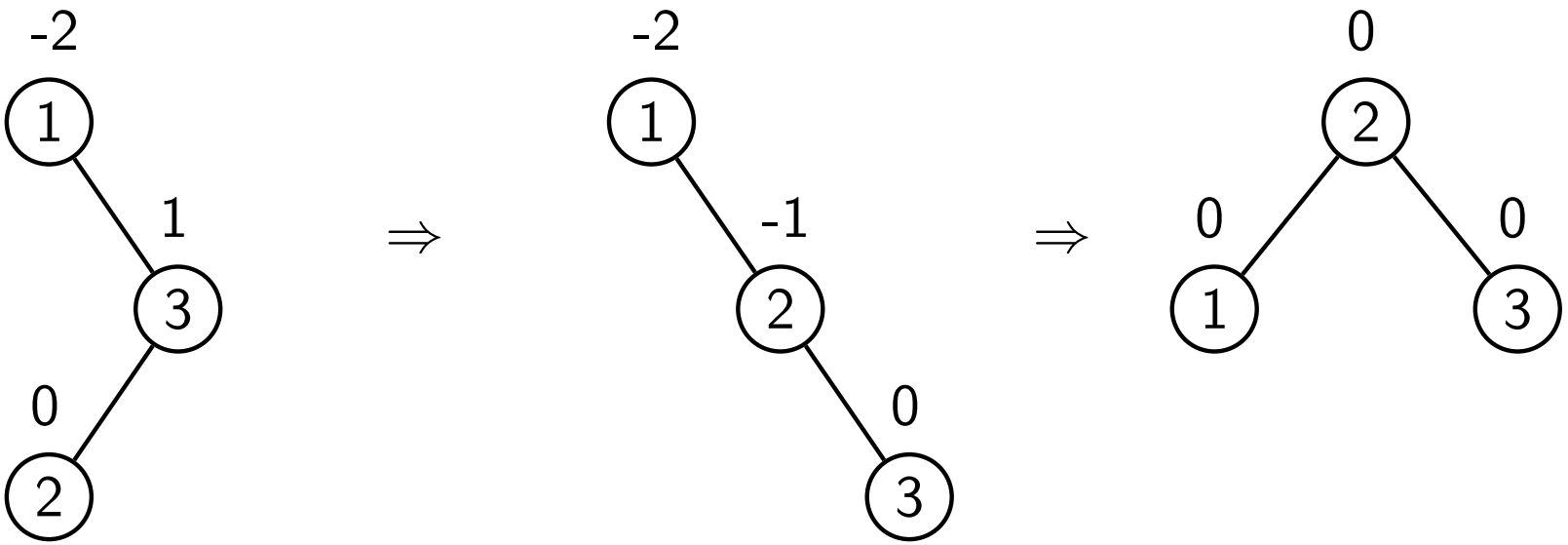
- an L-Rotation at the root of node 3's left sub-tree (i.e., node 1), and then
- an R-Rotation at node 3.



AVL Trees: RL-Rotation

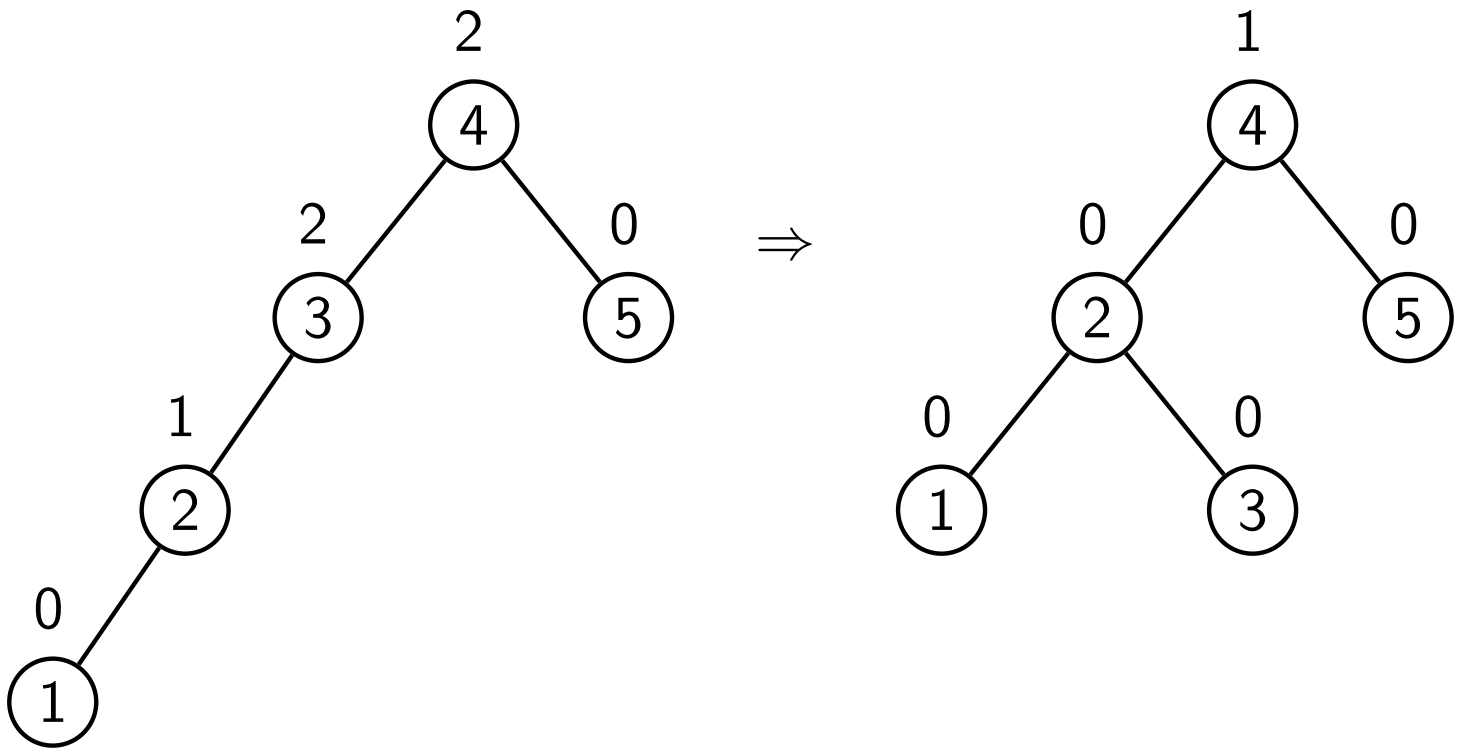
An RL-Rotation at node 1 performs:

- an R-Rotation at the root of node 1's right sub-tree (i.e., node 3), and then
- an L-Rotation at node 1.



AVL Trees: Where to Perform the Rotation

Along an unbalanced path, we may have several nodes with balance factor 2 (or -2):



It is always to start the re-balancing from the lowest unbalanced sub-tree.

AVL Trees: Which Type of Rotation to Perform

Let u be the new node just inserted to the AVL tree.

Think: What are the nodes that would possibly violate the balance condition?

After u's insertion, only the nodes on the path from the root to u can possibly have balance violation.

Let v be the *lowest* node that has balance violation.

There are only *four* possible *relative locations* between v and u:

- Left-Left Case: u is in the left sub-tree of the left child of v;
- Right-Right Case: u is in the right sub-tree of the right child of v;
- Left-Right Case: u is in the right sub-tree of the left child of v;
- Right-Left Case: u is in the left sub-tree of the right child of v

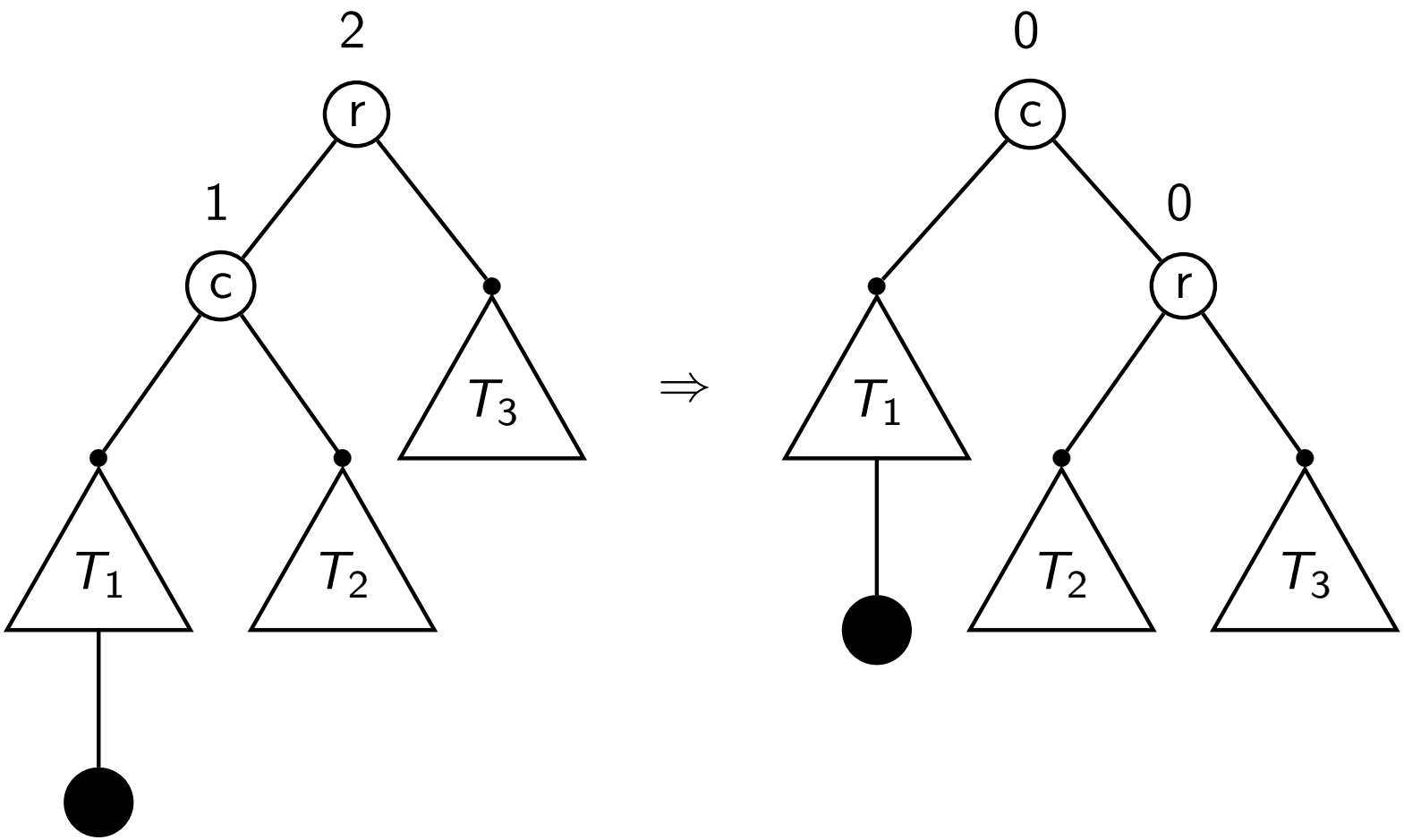
AVL Trees: Which Type of Rotation to Perform

The crucial idea of performing a rotation is to *rotate the "middle" (in terms of key values) node to be the "root"* among the three nodes involved in the rotation.

Consider the above four cases:

- Left-Left Case: perform an R-Rotation at v.
- Right-Right Case: perform an L-Rotation at v.
- Left-Right Case: perform an LR-Rotation at v.
- Right-Left Case: perform an RL-Rotation at v.

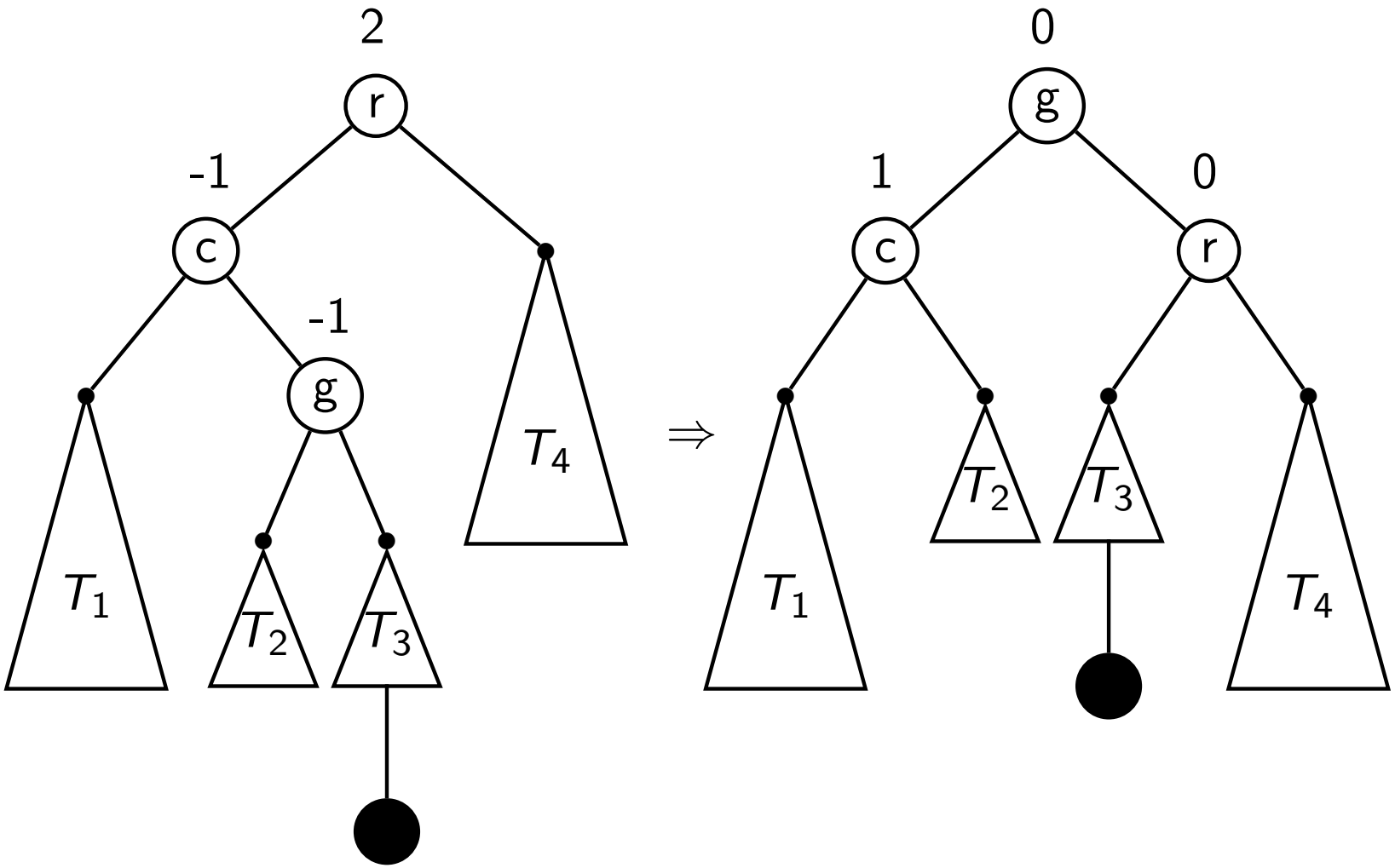
AVL Trees: The Single Rotation, Generally



This is a left-left case; an R-Rotation at r should be performed. An L-rotation is symmetric.

AVL Trees: The Double Rotation, Generally

This is an left-right case; an LR-Rotation should be performed:



This shows an LR-rotation; an RL-rotation is symmetric.

Properties of AVL Trees

The notion of "balance" that is implied by the AVL condition is sufficient to guarantee that the depth of an AVL tree with n nodes is $\Theta(\log n)$.

For random data, the depth is very close to $\log_2 n$, the optimum.

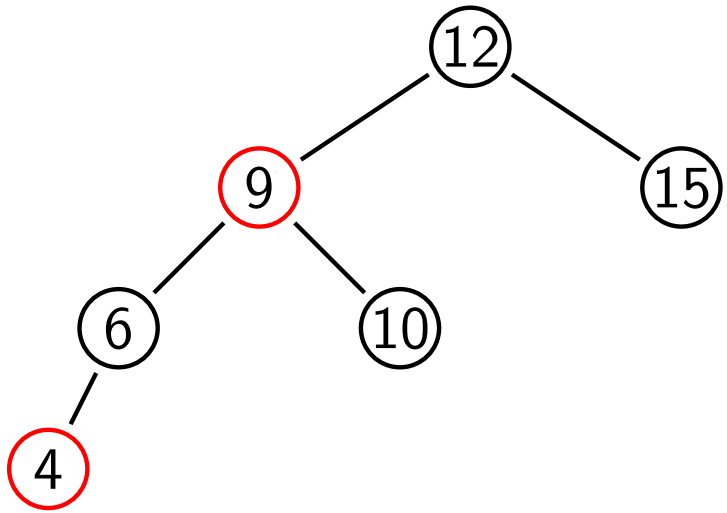
Some study shows that, in the worst case, search will need at most 45% more comparisons than with a perfectly balanced BST.

Deletion is harder to implement than insertion, but also $\Theta(\log n)$.

Other Kinds of Balanced Trees

A **red-black tree** is a BST with a slightly different concept of "balanced". Its nodes are coloured red or black, so that

1. No red node has a red child.
2. Every root-to-leaf path has the same number of black nodes.



A worst-case red-black tree (the longest path has at most twice of the nodes in the shortest path).

A **splay tree** is a BST which is not only self-adjusting, but also adaptive. Frequently accessed items are brought closer to the root, so their access becomes cheaper.

2–3 Trees

Recall that, in a binary search tree, each node u stores one and exactly one item (i.e., key).

And such an item serves as a *router*:

- when the search key $q = u.key$, return u;
- when $q > u.key$, it directs the search path to its right sub-tree; and
- when $q < u.key$, it directs the search path to its left sub-tree.

2–3 Trees

2–3 trees are (non-binary) search trees that allow *more than one* key but *no more than two* keys to be stored in a tree node.

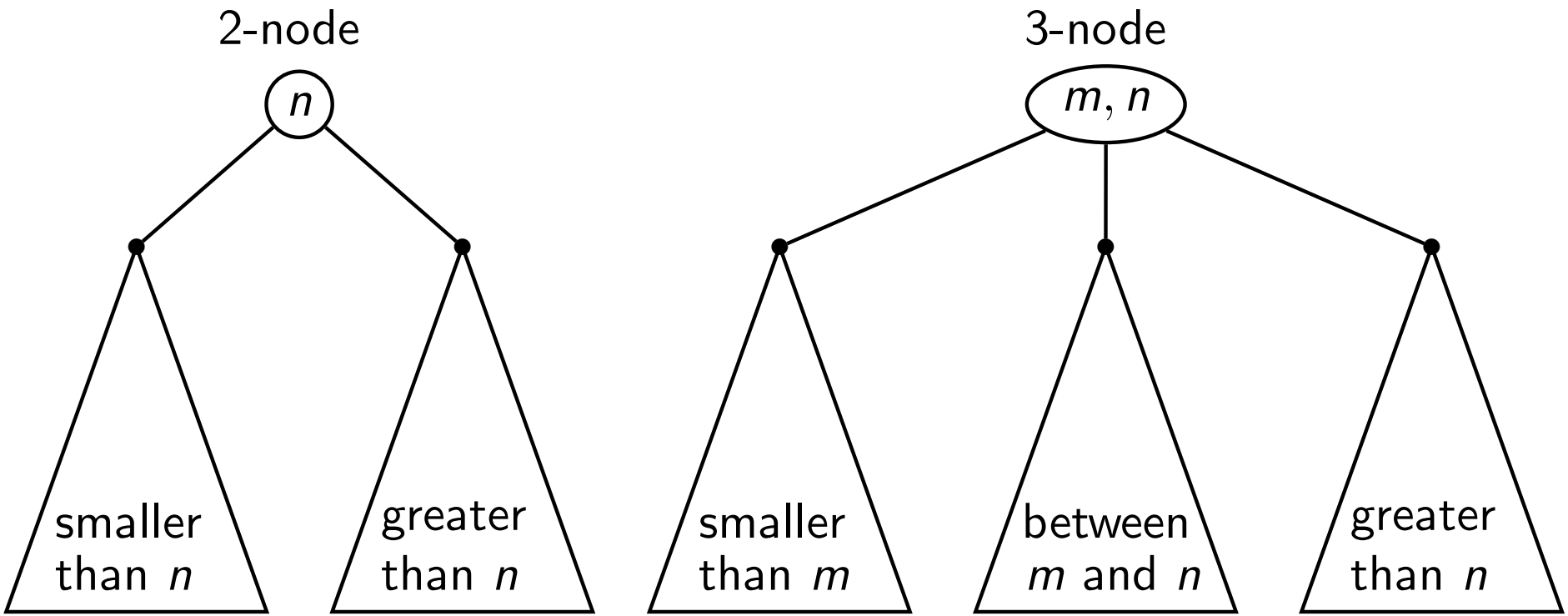
In other words, a **non-leaf node** u in a 2–3 tree can have either two children (when it stores one key) or three children (when it stores two keys).

Consider a 2–3 tree; we define:

- 2-node: a node u that holds **a single key**:
  - if u is an **internal node** (i.e., a non-leaf node), then u has *two* children;
  - otherwise, i.e., u is a leaf, then u has no child nodes.
- 3-node: a node that holds **two keys**:
  - if u is an **internal node**, then u has *three* children;
  - otherwise, u has no child nodes

Moreover, all the leaf nodes in a 2–3 tree are at the same level. In other words, a 2–3 tree is perfectly balanced.

2-Nodes and 3-Nodes



Insertion in a 2–3 Tree

To insert a key k, pretend that we are searching for k.

This will take us to a leaf node u in the tree, where k should now be inserted.

If u is a 2-node (currently containing only one key), then k can be inserted without further actions.

Otherwise, u is a 3-node (which already contains two keys), and the two inhabitants, together with k, make u "**overflow**":

In the sense that u have to store 3 keys which violates the requirement that each node can contain at most 2 keys.

Insertion in a 2–3 Tree

Suppose these prospective three keys stored in node u are $k_1 < k_2 < k_3$ in sorted order.

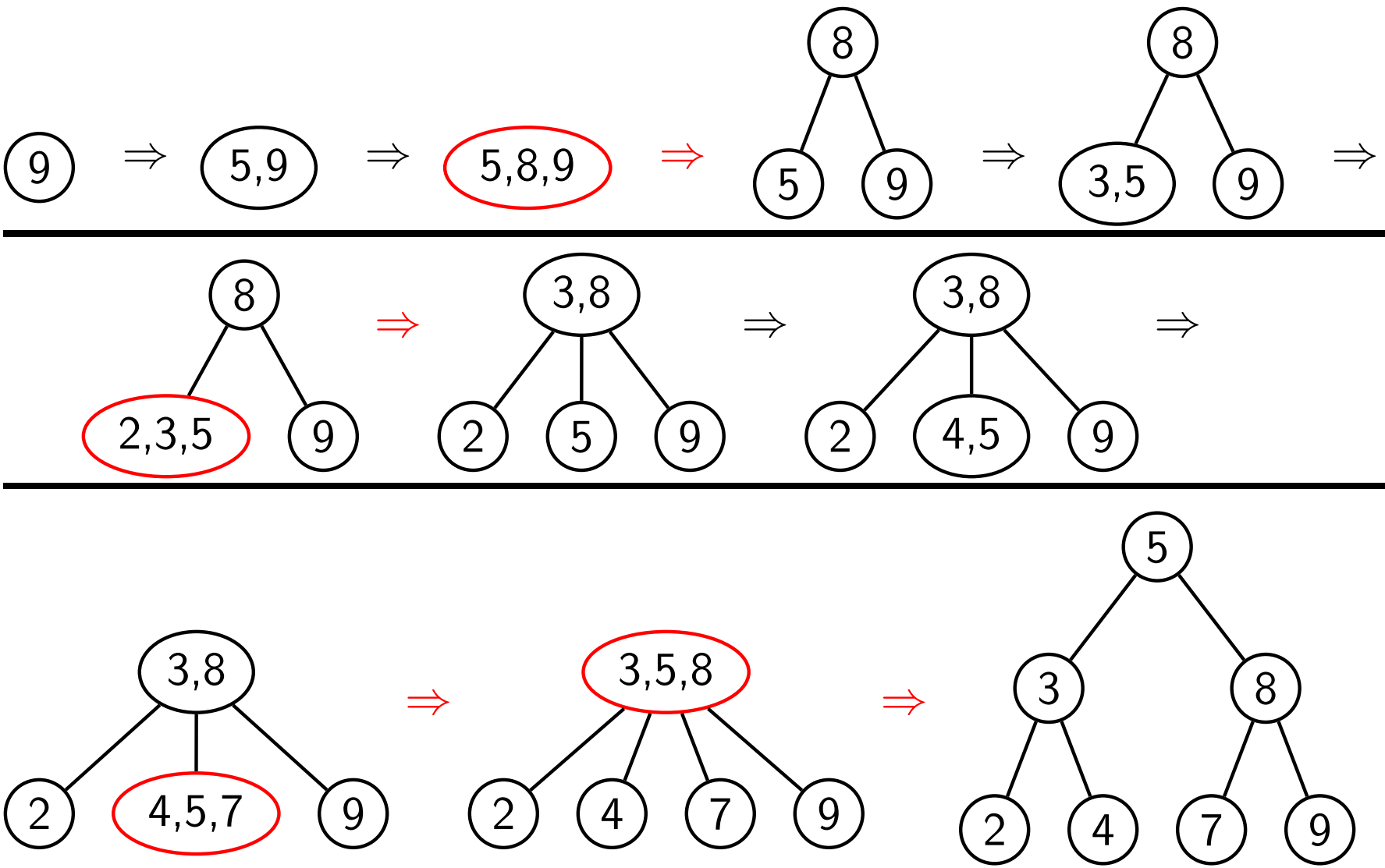
To resolve the overflowing of u, we need to *split* u, such that:

- k_1 and k_3 form their own individual 2-nodes, and
- the middle key, k_2, is *promoted* to the parent node of u.

The promotion may cause u's parent node to overflow, in which case the parent node gets split in the same way.

The only time the tree's height increases by 1 is when the root overflows. In this case, a new root is created.

Example: Build a 2–3 Tree from 9, 5, 8, 3, 2, 4, 7



Exercise: 2–3 Tree Construction

Build the 2–3 tree that results from inserting these keys, in the given order, into an initially empty tree:

C, O, M, P, U, T, I, N, G

2–3 Tree Analysis

Worst-case search time occurs when all the nodes are 2-nodes.

The relation between the number n of nodes and the height h is:

$$n = 1 + 2 + 4 + \ldots + 2^{h-1} = 2^h - 1$$

That is, $\log_2(n+1) = h$.

In the best case, all nodes are 3-nodes:

$$n = 2 + 2 \times 3 + 2 \times 3^2 + \ldots + 2 \times 3^{h-1} = 3^h - 1$$

That is, $\log_3(n+1) = h$.

Hence we have $\log_3(n+1) \leq h \leq \log_2(n+1)$.

Useful formula: $\sum_{i=0}^{n} a^i = \frac{a^{n+1}-1}{a-1}$, for $a \neq 1$.